Confidentiality of Messages

Elisabeth Oswald

ROADMAP

Security Notions for Confidentiality
- Symmetric Cryptography
- Adversaries
- Security Definitions

One-Time Symmetric Encryption
- Boolean Arithmetic
- One-time Pad
- Real vs Random Security

Symmetric Cryptography (recap)

Symmetric implies that the "same key" is used on "both ends" (i.e. encryption and decryption).

Encryption schemes are used to ensure confidentiality of messages.

Messages need to be represented as a 'number' (via some suitable encoding; we typically ignore this step). Any message can be thought of as an element from some message space. Encryption maps a message, with the use of a secret key, to some ciphertext.

- m is the message (plaintext) which is from the plaintext space: $m \in \mathcal{M}$
- c is the ciphertext which is from the ciphertext space: $c \in \mathcal{C}$
- sk is the secret key which is from the key space: $sk \in \mathcal{K}$

When helpful, I indicate secrets in red colour.

Symmetric Cryptography (recap), cont.

A symmetric encryption scheme $\mathcal{E}$ is given by three algorithms: *Gen*, *Enc*, and *Dec*.

- $Gen : \mathcal{K} \to sk$ generates a secret key via randomly sampling an element from the keyspace
- $Enc : \mathcal{M} \times \mathcal{K} \to \mathcal{C}$ maps an element from the plaintext space to the ciphertext space (using an element from the key space)
- $Dec : \mathcal{C} \times \mathcal{K} \to \mathcal{M}$ recovers the plaintext from the ciphertext (using the secret key)

A symmetric encryption scheme is said to be correct if:
$\forall m \in \mathcal{M}$ and $\forall sk \in \mathcal{K} : Dec_{sk}(Enc_{sk}(m)) = m$. Thus encryption must be invertible.

Notice that we refer to the encryption of "an element" (singular!). If we want to encrypt more than one element, we have to consider how this can/should be done securely. If a message consists of multiple elements, then vertical bars refer to the message length: say m is a message with multiple elements, then $n = |m|$ is the length of the message.

FUNCTIONS

Notice that in the previous slide I used arrows to refer to "processes" (like randomly choosing a key, encryption, and decryption). When an arrow is used in relation to some secret key, then it refers to the fact that there is random choice involved. When it is used between two (or more) spaces, then it indicates a mathematical function (aka map, mapping).

$Enc : \mathcal{M} \times \mathcal{K}$ is a function. It maps a pre-image m, k of its domain $(\mathcal{M}, \mathcal{K})$to the image $Enc(m, k)$. The image is contained in the codomain $\mathcal{C}$.

A function is injective if for $f(a) \neq f(b)$ for every two different pre-images.

A function is surjective if its image equals the codomain.

A function is called bijective if both injective and surjective (it maps exactly one element from its domain onto exactly one element in the codomain)

PROBABILITY DISTRIBUTIONS

A (discrete) probability distribution is a function that assigns a numerical value in $[0, 1]$ (a probability) to some event, and that obeys some rules: it assigns a probability $\Pr[]$ to each value of the sample space (finite domain); the sum of all assigned probabilities equals 1.

For instance: an event b might have two outcomes $\{0, 1\}$. Then a probability distribution could defined as $\Pr[b = 0] = 0.5$ and $\Pr[b = 1] = 0.5$; or as $\Pr[b = 0] = 0.4$ and $\Pr[b = 1] = 0.6$; but not as $\Pr[b = 0] = 0.5$ and $\Pr[b = 1] = 0.6$.

Because the sum of all probabilities add up to 1 we have that the probability of some $x \in X$ occuring $\Pr[x]$ is $1 - \Pr[\neg x]$ (so 1 minus the probability of x not occuring).

Suppose we roll a fair dice (6 sides), then the probability of 3 is $1/6$, and the probability of not getting a 3 (so either getting $1, 2, 4, 5, 6$) is $1 - 1/6 = 5/6$.

PROBABILITY DISTRIBUTIONS, CONT.

If we repeat an experiment multiple times, and each repetion is **independent** from all other experiments, then we can compute the probability for the "multiple times experiment" by simply **multiplying** the probabilities for the outcomes of each individual experiment.

If we perform two different experiments, and both are independently carried out, then we can compute the probability that both of them produce some specific outcome, by multiplying the probablities for their individual outcomes.

In the context of symmetric encryption we typically assume that all keys from the key space are chosen with equal probability:
The **uniform** probablity distribution over a finite set X assigns to each $x \in X$ the probability $\Pr[X = x] = 1/|X|$.

JOINT DISTRIBUTIONS

When considering encryption the properties of encryption functions, we wish to reason about the **joint distribution** of the inputs (message and key).

Because message and key are chosen independently, the joint probabability distribution $\Pr[M = m, K = k]$ is given as the product
$\Pr[M = m, K = k] = \Pr[M = m \wedge K = k] = \Pr[M = m] \cdot \Pr[K = k]$.

The joint distribution between plaintext and ciphertext, or ciphertext and key, typically have to be carefully derived, often using the concept of the conditional distribution.

In the security notion for confidentiality we concern ourselves with what an adversary learns about a message after seeing some ciphertext. This translates neatly to the concept of conditional distribution as well: $\Pr[M = m|C = c]$.

JOINT AND CONDITIONAL DISTRIBUTIONS, CONT.

Conditional probabilities are related to joint probablities via:

$$\Pr[M = m | C = c] = \frac{\Pr[M = m, C = c]}{\Pr[C = c]}. \tag{1}$$

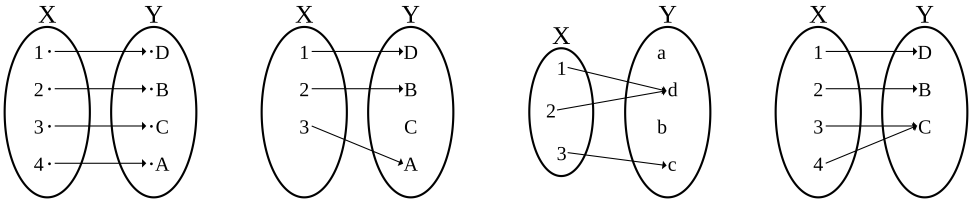
Bayes' theorem is useful at times:

$$\Pr[M = m | C = c] = \frac{\Pr[C = c | M = m] \cdot \Pr[M = m]}{\Pr[C = c]}. \tag{2}$$

Also the law of total probability applies;

$$\Pr[C = c] = \sum_{m} \Pr[C = c | M = m] \cdot \Pr[M = m] \tag{3}$$

OVER TO YOU



There are four pictures, each showing a function from the domain $\{1, 2, 3, 4\}$ to the co-domain $\{A, B, C, D\}$. Decide for each picture if it shows an injective, surjective, or bijective function, or if the function has none of these properties.

What property should an encryption/decryption function have?

OVER TO YOU

Assume that the distribution of a single symbol from the domain $\{0, 1\}$ is given as $\Pr[b = 0] = 0.4$ and $\Pr[b = 1] = 0.6$. And assume that in a string of such symbols, each symbol occurs independently of all other symbols in the string. What is the probability of the string "001" and what is the probability of "011".

Suppose that we roll a fair dice three times. What is the probability of having some value repeat?

Suppose that we roll a fair dice twice. What is the probability that both outcomes match?

Roadmap

Security Notions for Confidentiality
- Symmetric Cryptography
- Adversaries
- Security Definitions

One-Time Symmetric Encryption
- Boolean Arithmetic
- One-time Pad
- Real vs Random Security

How do we define "security"?

We want to design "secure" systems. But what does "secure" mean?

Security can only be made precise if we have attacks in mind: mostly we are trying to say that something is secure against certain types of attacks.

Listing types of attacks is possible but inefficient, and, it may invite "cat and mouse" games.

To ensure security against many types of attacks, it makes more sense to "capture" essential characteristics of these attacks via an **"adversarial model": this includes specifying what is assumed secret and what is assumed public, the goals of an adversary, the help that they get, and the computing power that they have**.

Kerckhoffs' Principle

You may have noticed that two variables are encoded in my slides using red font: the secret key and the message.

The purpose of encryption is obviously to protect the plaintext. Thus in practice we should assume that it is not known to an adversary.

The other item to be kept secret is the secret key. But notice that we assume that both encryption and decryption functions are supposed to be public knowledge. This may seem odd, but is a fundamental principle in modern cryptography.

Kerckhoffs' Principle

"The cipher method must not be required to be secret, and it must be able to fall into the hands of the enemy without inconvenience."

GOALS

In symmetric encryption the only secret parameters are the key and the message.

- Key recovery: given (many) ciphertexts, or, many pairs of plaintext and ciphertext, the adversary wants to work out the secret key (key recover, short KR). Once an adversary has the secret key they can read all messages that were ever encrypted with that key. They can also produce new ciphertexts.
- Plaintext recovery: given (many) ciphertexts, the adversary wants to recover the associated plaintexts. This may or may not give information about the secret key. This is often refered to as "one-way" (short OW).
- Partial plaintext recovery: given (some) ciphertext(s), the adversary wants to know "something" about the associated plaintext(s) (more often this is phrased as indistiguishability, short IND)

Clearly, key recovery implies plaintext recovery. But the opposite might not hold.
The goal of any symmetric encryption scheme is to protect the plaintext message, under the assumption that the key is secret, but the encryption/decryption process is public.

Help

In the context of symmetric encryption, an adversary may:

- be able to choose a message (for encryption) or a ciphertext (for decryption), and
- interact with an encryption/decryption oracle.

In the context of symmetric cryptography, the encryption and decryption oracle use the same secret key. Hence both give the same information and we make no distinction between them.

If an adversary cannot interact with an oracle, i.e. they only ever see a **single ciphertext** (perhaps a very long one), then we call this a **passive** adversary (PAS), or **one-time security (OTS)**, or an **eavesdropper** (EAV). (Different text books use different terms.)

If an adversary can interact with an oracle. i.e. they get multiple ciphertexts for the same key, then we call this a **chosen plaintext adversary** (short CPA).

When we will consider side channels, then the oracle will give some more information to the adversary.

Over to you

What might the following abbreviations refer to (referring to the concepts and abbreviations introduced in the two preceedings slides)?

- OW-PAS
- OW-CPA
- IND-PAS
- IND-CPA

Computing Power

In modern cryptography we consider adversaries with both unlimited computing power (akin to quantum adversaries) and adversaries with bounded computing power. This gives rise to:

Security classes

Perfect security. The adversary has unlimited computational powers yet cannot reach their goal.

Statistical security. The adversary has unlimited computational powers and they can reach their goal with an extremely small probability.

Computational security. The adversary has limited computational powers and within these constraints cannot reach their goal.

COMPUTING POWER, CONT.

Whenever we use the term “perfect” as part of a notion/definition, then we refer to computationally unbounded adversaries. In some textbooks/papers/websites, people also use the term “information theoretic” instead of “perfect”. **If a scheme is perfectly secret, then no adversary can ever break the scheme.**

If the term “perfect” is not used as part of a notion/definitions, then we refere to computationally bounded adversaries. We make it hard for them to win a game by choosing keys from a very large space. In this way we ensure that brute-force key search becomes practically infeasible. As a result, adversaries have to try other ways to win (perhaps by using some form of cryptanalysis). Consequently, cryptographic constructions are designed so that the effort of (known) cryptanalytic methods increases with the key space. **If a scheme is computationally secure, then we argue that no adversary can win with more than a “tiny probability”.**

ROADMAP

Security Notions for Confidentiality
- Symmetric Cryptography
- Adversaries
- Security Definitions

One-Time Symmetric Encryption
- Boolean Arithmetic
- One-time Pad
- Real vs Random Security

Definitions via Security Games: OW-PAS

The game is between an oracle $\mathcal{O}$ and an adversary $\mathcal{A}$.

Every time this game is played, the oracle and adversary perform the specified actions: choose a key, choose a target message, encrypt the target message, and send the resulting target ciphertext to the adversary.

The adversary outputs a message m', if this is equal to m^* they win.

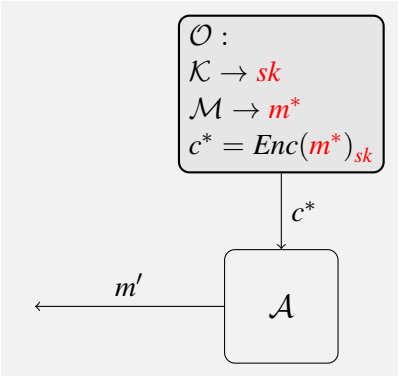


Figure: Exp ow-pas. A wins if $m' = m^*$.

An encryption scheme is called OW-PAS secure if no adversary can win this game with more than "a tiny" probability.

Definitions via Security Games: OW-CPA

In the CPA version of the OW game, the adversary can interact with the oracle.

Therefore a secret key and a target message are sampled once and placed into the oracle.

The adversary gets again a target ciphertext, but they can also interact with the oracle (dashed lines).

For computational security they can interact a limited number of times (bounded by the "security parameter").

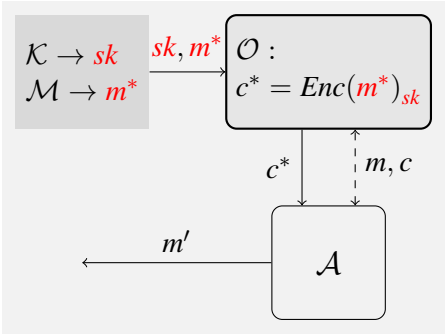


Figure: Exp ow-cpa. A wins if $m' = m^*$.

An encryption scheme is called OW-PAS secure if no adversary can win this game with more than "a tiny" probability. Obviously, for a computationally unbounded adversary, this game is easy to win.

INDISTINGUISHABILITY: IND-PAS

Indistinguishability: learn nothing (in addition to what is public knowledge) about a message when given its ciphertext.

The adversary is tasked to distinguish between the encryption of two chosen messages m_0, m_1 **that are exactly the input size of the encryption function**. The adversary outputs a bit b'.

The choice b is made once.

They win this game if $b' = b$.

A scheme is IND-CPA secure if no adversary can win this game with more than a "tiny probability".

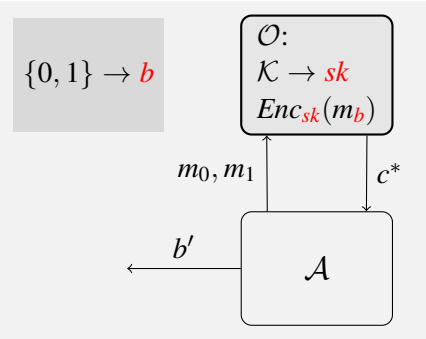


Figure: Exp ind-pas. Win if $b' = b$.

INDISTINGUISHABILITY: IND-CPA

The adversary is tasked to distinguish between the encryption of two chosen messages m_0, m_1 **of arbitrary but equal length**. The adversary outputs a bit b'.

The dashed lines visualise repeated messages between the adversary and the oracle. Like in the pictures before, the solid lines are messages that are sent exactly once in the game.

The choice of b is made once, as is the choice for sk.

They win this game if $b' = b$.

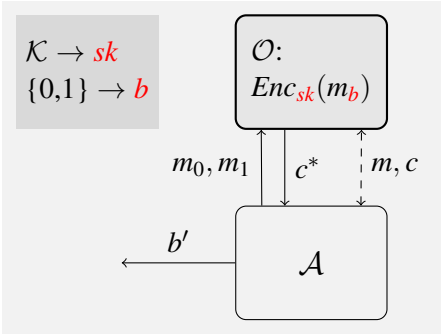


Figure: Exp ind-pas. Win if $b' = b$.

Left-Right Security Notion

An equivalent, but somewhat more modern, way of representing the idea of indistinguishability of ciphertexts is via the "left vs right world" notion.

The setup is the same.

The adversary sends and receives the same messages as before.

The win condition is the same.

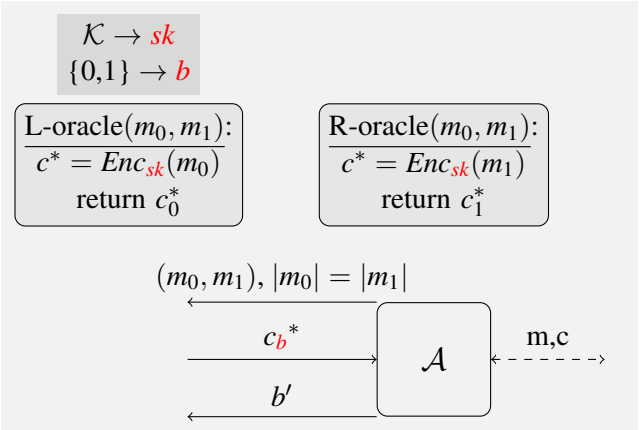


Figure: Exp LR-CPA (IND-CPA). Win if $b' = b$.

Your Turn

Question: Why do we require that the two messages have equal length?

Task: Draw the IND-PAS version of the game.

ROADMAP

Security Notions for Confidentiality
- Symmetric Cryptography
- Adversaries
- Security Definitions

One-Time Symmetric Encryption
- Boolean Arithmetic
- One-time Pad
- Real vs Random Security

Binary Numbers

We tend to work with digital computers, which represent all information as strings containing bits, e.g. the letter “a” typically is represented (using the ASCII code format) as “01100001”. Consequently, we often seek to encrypt bit-strings of some length.

Whilst such representations are “strings” (so sequences of characters of the alphabet $\{0,1\}$), they can also be interpreted as “numbers”, which enables computing on them.

So let us interprete $\{0,1\}$ as integers. We wish to compute with them, in such a way, that any operation gives a result again one of $\{0,1\}$.

We can do this using **modular arithmetic**.

Modular Arithmetic

For $x, n, \in \mathbb{Z}$, we say that n divides x (or x is a multiple of n), and write $n|x$, if there exists a $k \in \mathbb{Z}$ such that $x = kn$.

Given integers $a, b, N \in \mathbb{Z}$ we say that a is congruent to b modulo N

$$a \equiv b \pmod{N} \quad \text{iff} \quad N \text{ divides } b - a.$$

Often we are lazy and just write $a \equiv b$ if it is clear we are working modulo N.

Example: $16 \equiv 1 \pmod{5}$ since $1 - 16 = -3 \cdot 5$.
Example: $16 \equiv 0 \pmod{2}$ since $0 - 16 = -8 \cdot 2$.
For convenience we define the set

$$(\mathbb{Z}/N\mathbb{Z}) = \{0, \ldots, N-1\},$$

which is the set of remainders modulo N. Note: some authors use the shorthand $\mathbb{Z}_N = (\mathbb{Z}/N\mathbb{Z})$.

Modular Numbers

It is clear that for any modulus N, any integer $a \in \mathbb{Z}$ is congruent modulo N to an element in the set $\mathbb{Z}_N$.

This is because we can write

$$a = q \cdot N + r,$$

with $0 \leq r < N$ and clearly $a \equiv r \pmod{N}$.

Clearly we can do addition and multiplication in $\mathbb{Z}_N$.

- Addition
  - $(11 + 13) \pmod{16} \equiv 24 \pmod{16} \equiv 8 \pmod{16}$.
- Multiplication
  - $(11 \times 13) \pmod{16} \equiv 143 \pmod{16} \equiv 15 \pmod{16}$.

Modular Numbers, cont.

It is clear that we can also do subtraction (so the addition of the additive inverse); e.g. $11 - 13 \equiv -2 \equiv 14 \pmod{16}$. But what about division (aka multiplication with a multiplicative inverse)?
The multiplicative inverse of $a \in \mathbb{Z}_N$ is an element $b \in \mathbb{Z}_N$ with

$$a \cdot b \equiv b \cdot a \equiv 1.$$

We often write $b \equiv a^{-1}$.

Fact: $a \in \mathbb{Z}_N$ has a unique inverse modulo N iff a and N are relatively prime, i.e. $gcd(a, N) = 1$.

Notice that 2 is a prime number, so if we work $\pmod{2}$, then we can do all "basic arithmetic operations", and are guaranteed a result $\pmod{2}$.

Binary Modular Arithmetic: Arithmetic over $\mathbb{Z}_2$

Addition:

$$0 + 0 = 0$$
$$0 + 1 = 1$$
$$1 + 0 = 1$$
$$1 + 1 = 0$$

The addition modulo 2 corresponds to the action of the Boolean "exlusive-or" operator $\oplus$.

Multiplication

$$0 \cdot 0 = 0$$
$$0 \cdot 1 = 0$$
$$1 \cdot 0 = 0$$
$$1 \cdot 1 = 1$$

The multiplication modulo 2 corresponds to the action of the Boolean "and" operator $\wedge$.

The element 0 is the neutral element with respect to addition. The element 1 is the neutral element with respect to multiplication.

Bit Strings

The properties of addition modulo 2 lead to several useful facts about exclusive-or operations:

$$
\begin{aligned}
x \oplus x &= 0 \\
x \oplus 0 &= x \\
x \oplus 1 &= \bar{x} \ \text{(bitflip)} \\
x \oplus y &= y \oplus x \ \text{(symmetric)} \\
(x \oplus y) \oplus z &= x \oplus (y \oplus z) \ \text{(associative)}
\end{aligned}
$$

Addition modulo 2/exclusive-or can be naturally extended to bit-strings:
$00101 \oplus 01100 = 01001$.

Therefore these facts also apply to bit-strings.

We write $\{0,1\}^n$ to refer to the set of bit strings of fixed length n.

Roadmap

Security Notions for Confidentiality
- Symmetric Cryptography
- Adversaries
- Security Definitions

One-Time Symmetric Encryption
- Boolean Arithmetic
- One-time Pad
- Real vs Random Security

One-time Pad encryption

OTP

The encryption scheme OTP has a message space $\mathcal{M}$, a ciphertext space $\mathcal{C}$ and a key space $\mathcal{K}$ with $|\mathcal{K}| = |\mathcal{C}| = |\mathcal{M}| = \{0,1\}^n$.

Gen samples k uniformly at random from $\mathcal{K}$ for a one time use.

We define encryption as $c = Enc_k(m) = m + k \bmod 2$ (i.e. $+$ equals the $\oplus$ operator).

We define decryption likewise to be $m = Dec_k(c) = c + k \bmod 2$.

Correctness follows easily because
$Dec_k(Enc_k(m)) = (m \oplus k) \oplus k = m \oplus (k \oplus k) = m \oplus 0 = m$.
Notice that we use a key only a single time. Therefore if the same message is repeatedly encrypted, each encryption will result in a different ciphertext.

EXAMPLES

$\Pr[c]$	k	$c = 01 \oplus k$
1/4	00	01
1/4	01	00
1/4	10	11
1/4	11	10

$\Pr[c]$	k	$c = 10 \oplus k$
1/4	00	10
1/4	01	11
1/4	10	00
1/4	11	01

$\Pr[c]$	k	$c = 11 \oplus k$
	00	
	01	
	10	
	11	

Here are three tables that show the ciphertext distribution upon encrypting bit-strings of length 2. What do you notice in relation to the distribution of ciphertexts?

Ciphertext distribution of the OTP

For any $\mathcal{M}$, the distribution of ciphertexts after OTP encryption is the uniform distribution: $\Pr[C = c] = 1/2^n$.

Proof argument: we take some pair m, c. Because $c = Enc_k(m) = m \oplus k$, the choice m, c fixes some value $k = m \oplus c$ (OTP encryption is bijective).

The probability of having a ciphertext c for some fixed m is
$\Pr[C = c|M = m] = \Pr[c = k \oplus m] = \Pr[k = c \oplus m]$.

In the OTP scheme keys are selected from the uniform distribution, therefore
$\Pr[k = c \oplus m] = \Pr[K = k] = 1/2^n$.

This implies that $\Pr[C = c|M = m] = \Pr[K = k] = 1/2^n$, and therefore
$\Pr[C = c] = \sum_m \Pr[C = c|M = m] \cdot \Pr[M = m] = \sum_m 1/2^n \cdot \Pr[M = m] = 1/2^n$.

PERFECT SECRECY

Perfect secrecy

We call an encryption scheme perfectly secret if an adversary with unlimited computing power learns nothing about the message given its ciphertext: $\Pr[M = m] = \Pr[M = m | C = c]$.

OTP encryption is perfectly secret.

$$\begin{aligned}\Pr[M = m | C = c] &= \frac{\Pr[C = c | M = m] \cdot \Pr[M = m]}{\Pr[C = c]} \\ &= \frac{2^{-n} \cdot \Pr[M = m]}{2^{-n}} \\ &= \Pr[M = m]\end{aligned}$$

OTP Ciphertexts give no information about plaintexts

If an adversary sees outputs of an encryption oracle that implements the OTP, then, the distribution of ciphertexts will be uniform.

We used this fact to demonstrate perfect secrecy: no matter how much computing power the adverary throws at the problem, they will never learn anything more about the message given a ciphertext.

So the fact that ciphertexts are uniformly distributed implies that they carry “no information about the plaintext”.

Or: the adversary cannot distinguish between an oracle that implements OTP encryption, or an oracle that just outputs random strings.

Roadmap

Security Notions for Confidentiality
- Symmetric Cryptography
- Adversaries
- Security Definitions

One-Time Symmetric Encryption
- Boolean Arithmetic
- One-time Pad
- Real vs Random Security

Real vs Random Security

The mental model in the RoR security game is as simple as in the LoR security game.

There are two “worlds”.

The “real world” implements some encryption scheme (the one that we wish to prove secure). It has some well defined inputs, samples/has the key according to the specification of the scheme, and produces a faithful output.

The “random world” has no access to the key. Therefore it cannot implement the encryption function, and it produces random output instead.

If it is possible to come up with a “random world” implementation that is indistinguishable from the real world implementation, then the output of the real word is effectively random and therefore carries no information about the secrets inside the “real world”.

Real vs Random Security for the OTP

The goal of the adversary in the RoR game is to distinguish between outputs that come from the real world and outputs that come from the random world.

We allow the adversary to select a message $m \in \mathcal{M}$ (with some fixed length n). Remember though that this is still a "one-time" encryption, so they get a single ciphertext.

The real world implements OTP encryption.

The "random world" returns values from a uniform distribution.

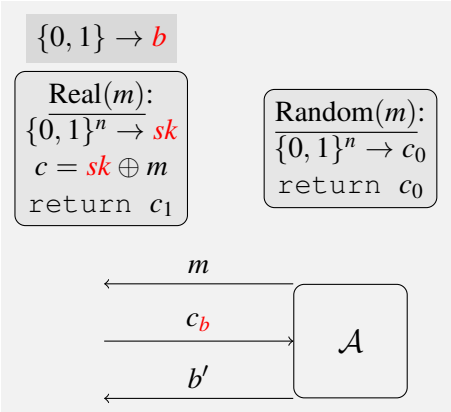


Figure: Exp RR-Pas. Win if $b' = b$.

Over to you

Draw the LoR security game for the OTP. Can you argue why no adversary can ever win the LoR version?